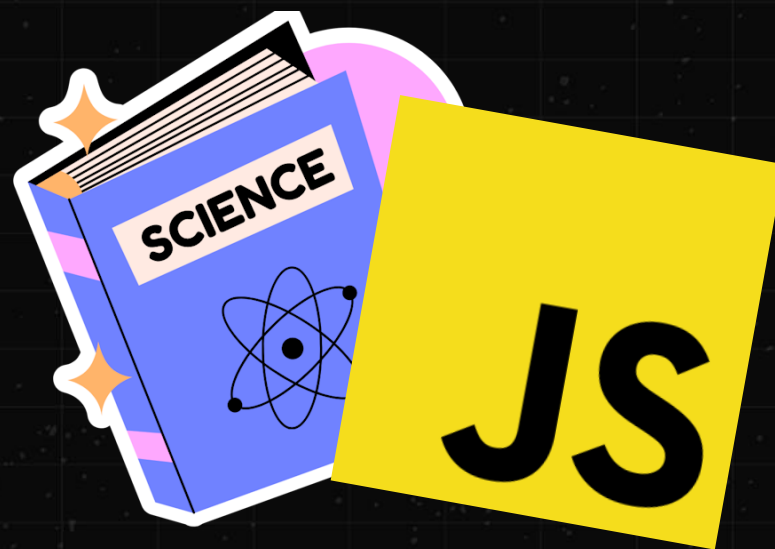


ЛЕКЦИЯ #20

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Продолжаем говорить про **поиск в строке**
- * А также про **парсинг и валидацию структуры** с помощью грамматик
- * Знакомимся с **регулярными выражениями**

А ЧТО ТАМ **БЫЛО**
С ПОИСКОМ В СТРОКЕ?



ПОИСК В СТРОКЕ – ЭТО НЕ ПРОСТО

- * Так как мы ищем не «один» символ, а последовательность символов
- * Наивные алгоритмы здесь крайне неэффективны
- * Часто нам нужно искать не конкретную подстроку, а строку, похожую на некоторый «шаблон»
- * Например, есть строка «Тема: покупка лицензии. Дата: 21.10.2019»
- * И нам нужно извлечь слова после «Тема:» и «Дата:»

НО КАК ОПИСАТЬ ТАКОЙ
ЗАПРОС НА ПОИСК?



НАМ НУЖЕН DSL

- * Domain-Specific Language (DSL) — это такой язык, который **заточен на решение конкретной задачи**
- * Например, [SQL](#) или [HTML](#)
- * Такой язык **учитывает нюансы предметной области** и предоставляет наиболее удобный для неё инструментарий
- * Для **задач поиска/замены/сопоставления** строк тоже есть свой DSL — регулярные выражения

РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ

- * Это DSL, который позволяет нам описать грамматику текста, используя регулярные правила
- * Например, строки вида «№ 123456» и «№ 642» можно описать грамматикой «№ \d+»
- * \d – специальный символ, означающий арабскую цифру от 0 до 9
- * + – квантификатор, означающий, что таких цифр может быть от 1 до бесконечности

А ПРИ ЧЕМ **ТУТ**
ГРАММАТИКИ?



ЧТО ТАКОЕ ЯЗЫК?

- * Язык определяется алфавитом букв
- * Из букв согласно правилам языка образуются слова
- * Из слов согласно правилам языка могут выстраиваться предложения
- * Грамматика – это формализованное описание языка и его правил
- * То есть любой язык задается грамматикой

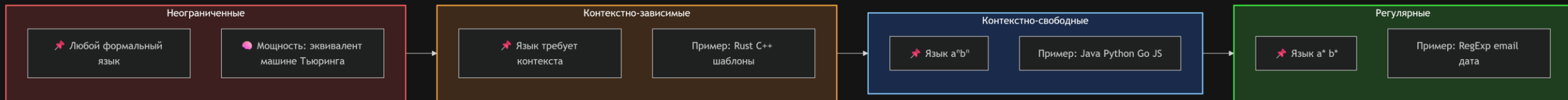
ПОЛУЧАЕТСЯ
У КАЖДОГО ЯЗЫКА ЕСТЬ
СВОЯ ГРАММАТИКА?



ДА

- * Но **некоторые грамматики очень сложны**, чтобы их можно было просто описать в виде чётких правил
- * Выделяют **четыре базовых класса грамматик** (иерархия Хомского)
- * Неограниченные (тип 0)
- * Контекстно-зависимые (тип 1)
- * Контекстно-свободные (тип 2)
- * Регулярные (тип 3)

ВИЗУАЛИЗИРУЕМ





```
<Float>    ::= <Sign>? <Integer> <Fraction>? <Exponent>?  
            | <Sign>? <Integer>? <Fraction> <Exponent>?
```

```
<Sign>      ::= "+" | "-"
```

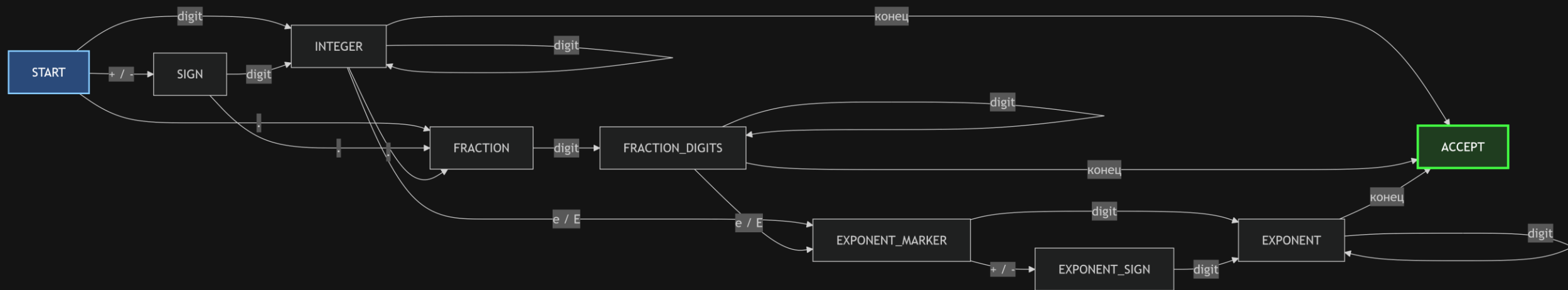
```
<Integer>   ::= <Digit>+
```

```
<Fraction>  ::= "." <Digit>+
```

```
<Exponent> ::= ("e" | "E") <Sign>? <Digit>+
```

```
<Digit>     ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

АВТОМАТ СТРОИТСЯ ПО ГРАММАТИКЕ



second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * Надо понять, что **грамматика** — это своего рода «**метаязык**» (язык для описания языков)
- * Более сложные классы грамматик предоставляют более богатые выразительные возможности для описания языков
- * С другой стороны, **их сложнее реализовать** (парсить)
- * В программировании наиболее популярные классы грамматик — это регулярные и контекстно-свободные

КОНТЕКСТНО-СВОБОДНЫЕ ГРАММАТИКИ

- * Разбиение грамматики на отдельные части (продукции)
- * Более сложные части могут строиться из более простых
- * Есть **поддержка рекурсии** на правилах
- * Языки, описанные контекстно-свободной грамматикой, распознаются КА со стековой (магазинной) памятью

```
<json>      ::= <value>

<value>      ::= <object> | <array> | <string> | <number> | "true" | "false" | "null"

<object>     ::= "{" <members> "}" | "{" "}"
<members>    ::= <pair> | <pair> "," <members>
<pair>       ::= <string> ":" <value>

<array>      ::= "[" <elements> "]" | "[" "]"
<elements>   ::= <value> | <value> "," <elements>

<string>     ::= '"' <chars> '"' | "'" <chars> "'"
<chars>      ::= <char> | <char> <chars>
<char>       ::= <any-unicode> | <escape>
<any-unicode> ::= <all-symbols-except-quote-and-backslash>

<escape>     ::= "\" ( '"' | "\" | "/" | "b" | "f" | "n" | "r" | "t" | "u" <hex> <hex> <hex> <hex> )
<hex>        ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" | "A" | "B" | "C" | "D" | "E" | "F"

<number>     ::= <integer> <fraction> <exponent> | <integer> <fraction> | <integer> <exponent> | <integer>
<integer>    ::= "0" | <non-zero-digit> <digits> | "-" "0" | "-" <non-zero-digit> <digits>
<digits>     ::= <digit> | <digit> <digits>
<digit>      ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
<non-zero-digit> ::= "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
<fraction>   ::= "." <digits>
<exponent>   ::= "e" <exponent-sign> <digits> | "E" <exponent-sign> <digits> | "e" <digits> | "E" <digits>
<exponent-sign> ::= "+" | "-"
```

ЧТО ЗА КА СО
СТЕКОВОЙ ПАМЯТЬЮ?



НИЧЕГО СЛОЖНОГО

- * При реализации простого конечного автомата его состояние — это переменная или свойство `state`, которое **при каждом переходе перезаписывается**
- * У автомата со стековой памятью (магазинный автомат) состояние — **это стек**
- * Это позволяет решать проблемы для грамматик вида `"{" <members> "}"`
- * То есть символ `"}"` допустим, только если ему предшествовал `"{"`

```
function checkBalance(str) {
  const stack = [];

  const brackets = {
    "(": ")",
    "[": "]",
    "{": "}",
    "<": ">",
  };

  // Множество открывающих скобок (для быстрой проверки)
  const closeBrackets = new Set([")", "]", "}", ">"]);

  for (const char of str) {
    // Если символ — открывающая скобка
    if (char in brackets) {
      stack.push(brackets[char]);
      continue;
    }

    // Если символ — закрывающая скобка
    if (closeBrackets.has(char)) {
      // Проверяем, соответствует ли закрывающая скобка последней открытой
      if (stack.pop() !== char) {
        return false;
      }
    }
  }

  // Если стек пуст — все скобки закрыты корректно
  return stack.length === 0;
}
```

```
console.log(checkBalance("{()}")); // ✓ true
console.log(checkBalance("{[]})")); // ✗ false
console.log(checkBalance("{}[]<>")); // ✓ true
console.log(checkBalance("{}"); // ✗ false
console.log(checkBalance("")); // ✓ true
console.log(checkBalance("{[<>]}")); // ✓ true
console.log(checkBalance("{[>]}")); // ✗ false
```

РЕФЛЕКСИРУЕМ

- * Благодаря стековой памяти мы можем создать такой автомат, который **сможет обрабатывать рекурсии**
- * Это легко осознать, особенно если вспомнить, что стек используется функциями, в том числе для поддержки рекурсивных вызовов
- * Автомат без магазинной памяти может обрабатывать **только простые грамматики, где рекурсия не нужна**
- * Например, парсинг числа с плавающей точкой
- * Такие грамматики называются регулярными

```

function parseNumber(str) {
  const states = {
    START: 0,
    SIGN: 1,
    INTEGER: 2,
    DECIMAL_POINT: 3,
    FRACTION: 4,
    DONE: 5,
    ERROR: 6,
  };

  let state = states.START;
  let result = "";
  let i = 0;
  const n = str.length;

  while (i < n && state !== states.ERROR && state !== states.DONE) {
    const ch = str[i];

    switch (state) {
      case states.START:
        if (ch === " " || ch === "\t" || ch === "\n") {
          i++;
          continue;
        } else if (ch === "-" || ch === "+") {
          result += ch;
          state = states.SIGN;
          i++;
        } else if (ch >= "0" && ch <= "9") {
          result += ch;
          state = states.INTEGER;
          i++;
        } else if (ch === ".") {
          result += "0.";
          state = states.FRACTION;
          i++;
        } else {
          state = states.ERROR;
        }
        break;

```

```

      case states.SIGN:
        if (ch >= "0" && ch <= "9") {
          result += ch;
          state = states.INTEGER;
          i++;
        } else if (ch === ".") {
          result += "0.";
          state = states.FRACTION;
          i++;
        } else {
          state = states.ERROR;
        }
        break;

      case states.INTEGER:
        if (ch >= "0" && ch <= "9") {
          result += ch;
          i++;
        } else if (ch === ".") {
          result += ch;
          state = states.DECIMAL_POINT;
          i++;
        } else {
          state = states.DONE;
        }
        break;

      case states.DECIMAL_POINT:
        if (ch >= "0" && ch <= "9") {
          result += ch;
          state = states.FRACTION;
          i++;
        } else {
          state = states.ERROR;
        }
        break;

```

```

      case states.FRACTION:
        if (ch >= "0" && ch <= "9") {
          result += ch;
          i++;
        } else {
          state = states.DONE;
        }
        break;

      default:
        state = states.ERROR;
    }

    if (state === states.ERROR || result === "") {
      return NaN;
    }

    return Number(result);
  }
}

```

РЕГУЛЯРНЫЕ ГРАММАТИКИ

- * Самые примитивные грамматики
- * Они порождают **только один символ на каждом шаге**
- * Языки, описанные такой грамматикой, распознаются обычным конечным автоматом
- * Классические регулярные выражения (без обратных ссылок и рекурсии) **описываются регулярной грамматикой и распознаются КА**

НЕСМОТРЯ НА ПРИМИТИВНОСТЬ

- * Регулярные грамматики отлично подходят для описания целого класса задач: лексический анализ, поиск по шаблону, валидация форматов
- * Например, проверка email или номера телефона — всё это задачи регулярных грамматик
- * Они просты в реализации и очень эффективны
- * Однако они не могут описывать вложенные структуры и сбалансированные скобки

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * Сейчас разберёмся на практике, и станет всё понятно
- * А пока запомните: **синтаксис большинства языков программирования** нельзя описать регулярной грамматикой
- * Хотя некоторые языки ассемблера могут быть исключением
- * Большинство языков программирования описываются **контекстно-свободной грамматикой** или даже **контекстно-зависимой**
- * Более сложные грамматики могут использоваться для описания «человеческих» (естественных) языков

ТАК ЧТО ТАМ С
РЕГУЛЯРНЫМИ
ВЫРАЖЕНИЯМИ?



РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ В JS

- * Класс `RegExp`
- * Поддерживается литеральная запись с помощью символов `/.../`
- * У экземпляров `String` есть множество методов, которые принимают регулярные выражения
- * У экземпляров `RegExp` тоже есть свои методы
- * Скомпилировать регулярное выражение можно с разными флагами

ПОИСК ПОДСТРОКИ

- * `RegExp.prototype.exec`
- * `String.prototype.search`
- * `String.prototype.match`
- * `String.prototype.matchAll`



```
// 1. exec — поиск в строке (по одному совпадению)
const text1 = "The price is 100 dollars and 200 euros.";
const regex1 = /\d+/g;

let match1 = regex1.exec(text1);

while (match1 !== null) {
  console.log(`Найдено: ${match1[0]} на позиции ${match1.index}`);
  // exec запоминает позицию (если у regex есть флаг g)
  match1 = regex1.exec(text1);
}

// Найдено: 100 на позиции 13
// Найдено: 200 на позиции 30

// 2. search — индекс первого совпадения
const index = "Hello, world! Hello, everyone!".search(/world/);
console.log(`"world" найден на позиции: ${index}`); // 7

// 3. match — массив совпадений
const matches = "The price is 100 dollars and 200 euros.".match(/\d+/g);
console.log(matches); // ["100", "200"]
```



```
// 4. matchAll — итератор по всем совпадениям (с группами)
const iterator = "Today is 2025-01-15 and tomorrow is 2025-01-16."
  .matchAll(/(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/g);

for (const match of iterator) {
  console.log(`Дата: ${match[0]}`);
  console.log(`Год: ${match.groups.year}`);
  console.log(`Месяц: ${match.groups.month}`);
  console.log(`День: ${match.groups.day}`);
}

// Дата: 2025-01-15
// Год: 2025
// Месяц: 01
// День: 15
// Дата: 2025-01-16
// Год: 2025
// Месяц: 01
// День: 16
```

ПОИСК И ЗАМЕНА ПОДСТРОКИ

- * `String.prototype.replace`
- * `String.prototype.replaceAll`
- * `String.prototype.split`



// 1. replace — замена по шаблону (первое совпадение или все с флагом g)

// Простая замена (только первое совпадение)

```
const result1 = "The price is 100 dollars and 200 euros.".replace(/\d+/, "XXX");  
console.log(result1); // "The price is XXX dollars and 200 euros."
```

// Замена всех совпадений (с флагом g)

```
const result2 = "The price is 100 dollars and 200 euros.".replace(/\d+/g, "XXX");  
console.log(result2); // "The price is XXX dollars and XXX euros."
```

// Замена с группами (захват частей)

```
const result3 = "2025-01-15".replace(/(\d{4})-(\d{2})-(\d{2})/, "$3.$2.$1");  
console.log(result3); // "15.01.2025"
```

// 2. replaceAll — замена ВСЕХ совпадений

// replaceAll с регулярным выражением (флаг g обязателен)

```
const result4 = "cat dog cat bird cat".replaceAll(/cat/g, "🐱");  
console.log(result4); // "🐱 dog 🐱 bird 🐱"
```

// 3. split — разбиение строки по разделителю

```
const parts = "apple, banana; orange: grape".split(/[,:;\s*]/);  
console.log(parts); // ["apple", "banana", "orange", "grape"]
```

ПРОВЕРКА НА СООТВЕТСТВИЕ ШАБЛОНУ

* RegExp.prototype.test

```
// 4. Проверка email
const emailRegex = /^[^@]+@[^@]+\.[^@]+$/;
console.log(emailRegex.test("user@example.com")); // true
console.log(emailRegex.test("a@b.c")); // true
console.log(emailRegex.test("invalid-email")); // false

// 5. Проверка номера телефона
const phoneRegex = /^[\+]?[0-9]{1,3}[\s-]?([0-9]{1,4})?[\s-]?[0-9]{1,4}[\s-]?[0-9]{1,4}[\s-]?[0-9]{1,4}$/;
console.log(phoneRegex.test("+7 (123) 456-78-90")); // true
console.log(phoneRegex.test("123-456")); // true (простой номер)
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЖКА, СТОЖКААА!

НАЧНЕМ С ПРОСТОГО

- * Как описать несколько подстрок?
- * Введем символ альтернации |
- * Андрей | Даша | Степан
- * Порядок имеет значение

АЛЬТЕРНАЦИЯ ВНУТРИ ПОДСТРОКИ

- * Введем специальные символы группировки ()

- * Ру(ка | кав)

- * То же самое, что и Рука | Рукав

- * Можно совмещать

- * Ру(ка | кав) | Штаны

НАБОР СИМВОЛОВ

- * Если мы не знаем точно символ, который ищем, то **можно указать множество его значений**
- * Введем специальные символы для такого множества []
- * [абв123]
- * Один символ может быть а, б, в, 1, 2 или 3
- * [СБ]ага → Сага | Бага
- * [СБ][ае]га → Сага | Бага | Сега | Бега

НАБОР ИСКЛЮЧАЮЩИХ СИМВОЛОВ

- * Обычный набор говорит, **из каких символов** может состоять один конкретный символ
- * Можно задать, из каких символов он **не может состоять**
- * Введем специальный символ отрицания [^]
- * `[^0123456789]`
- * Любые символы, **кроме арабских цифр**

ДИАПАЗОН СИМВОЛОВ

- * Внутри набора символов **можно использовать диапазоны из Юникода**
- * `[a-zA-Z0-9]`
- * `[^а-я]`
- * `[😄 - 😂]` (нужен специальный флаг `u`)
- * Нюансы `[а-яё]`

СПЕЦСИМВОЛЫ

* Для удобства для ряда символов введем псевдонимы

* `\n` – перевод строки

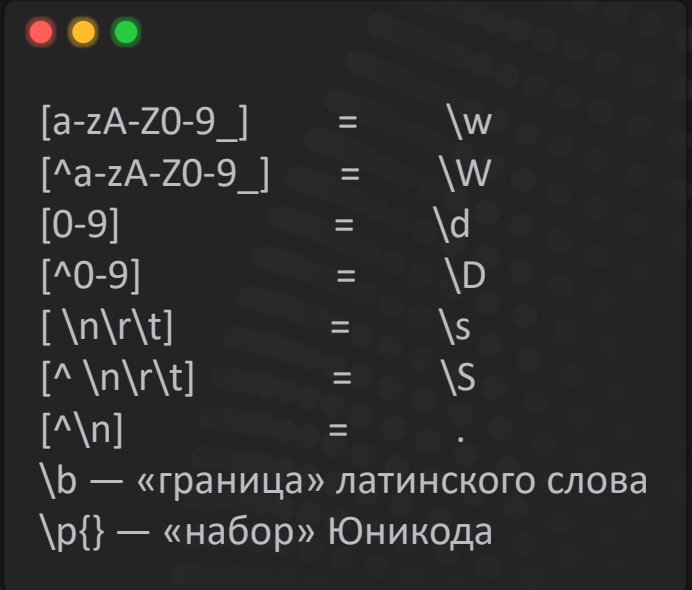
* `\r` – возврат каретки

* `\t` – табуляция

* `\v` – вертикальная табуляция

СИМВОЛЬНЫЕ КЛАССЫ

✱ Для популярных наборов есть специальные псевдонимы

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. It contains a list of regex pseudonyms and their corresponding character classes, followed by two explanatory lines.

```
[a-zA-Z0-9_]      =      \w
[^a-zA-Z0-9_]    =      \W
[0-9]            =      \d
[^0-9]           =      \D
[ \n\r\t]        =      \s
[^ \n\r\t]       =      \S
[^\n]            =      .
\b — «граница» латинского слова
\p{ } — «набор» Юникода
```

ИСПОЛЬЗОВАНИЕ СПЕЦСИМВОЛОВ КАК ОБЫЧНЫЕ

- * Чтобы сделать любой спецсимвол обычным, просто добавим перед ним \ – экранирование
- * Б\(\ага\|аба\)
- * \ можно тоже экранировать
- * \\
- * Внутри набора символов главное экранировать] \ и -
- * [[\]()|\-\\]

ЯКОРНЫЕ ССЫЛКИ

- * Как сказать, что символ должен быть в начале или конце строки?
- * Введем специальные символы для обозначения начала и конца
- * ^ — начало
- * \$ — конец
- * ^Привет
- * Пока\$
- * ^Привет и пока\$

КВАНТИФИКАТОРЫ

- * Использовать одинаковые наборы символов подряд неудобно
- * `[a-я][a-я][a-я][a-я]`
- * Введем специальные символы, показывающие количество повторений
- * `[a-я]{4}`
- * `[a-я]{2,3}`
- * `[a-я]{4,}`

ЕЩЕ ПРО КВАНТИФИКАТОРЫ

- * Квантификаторы можно применить к любому символу
- * $a\{4\}$
- * Или группе
- * $Py(ka | kav)\{0,1\}$

ДОПОЛНИТЕЛЬНЫЕ КВАНТИФИКАТОРЫ

✱ Для ряда популярных квантификаторов есть «сахар»

✱ $[a-y]\{0,\}$ $\rightarrow$ $[a-y]^*$

✱ $[a-y]\{1,\}$ $\rightarrow$ $[a-y]^+$

✱ $[a-y]\{0,1\}$ $\rightarrow$ $[a-y]^?$

ЖАДНОСТЬ КВАНТИФИКАТОРОВ

- * Квантификаторы * и + пытаются найти подстроку с **максимальным включением**
- * `[a-z ]+end`
- * `Begin end end end end end`
- * Введем специальный символ, чтобы **найти минимальную подстроку**
- * `[a-z ]+?end`
- * `Begin end end end end end`

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

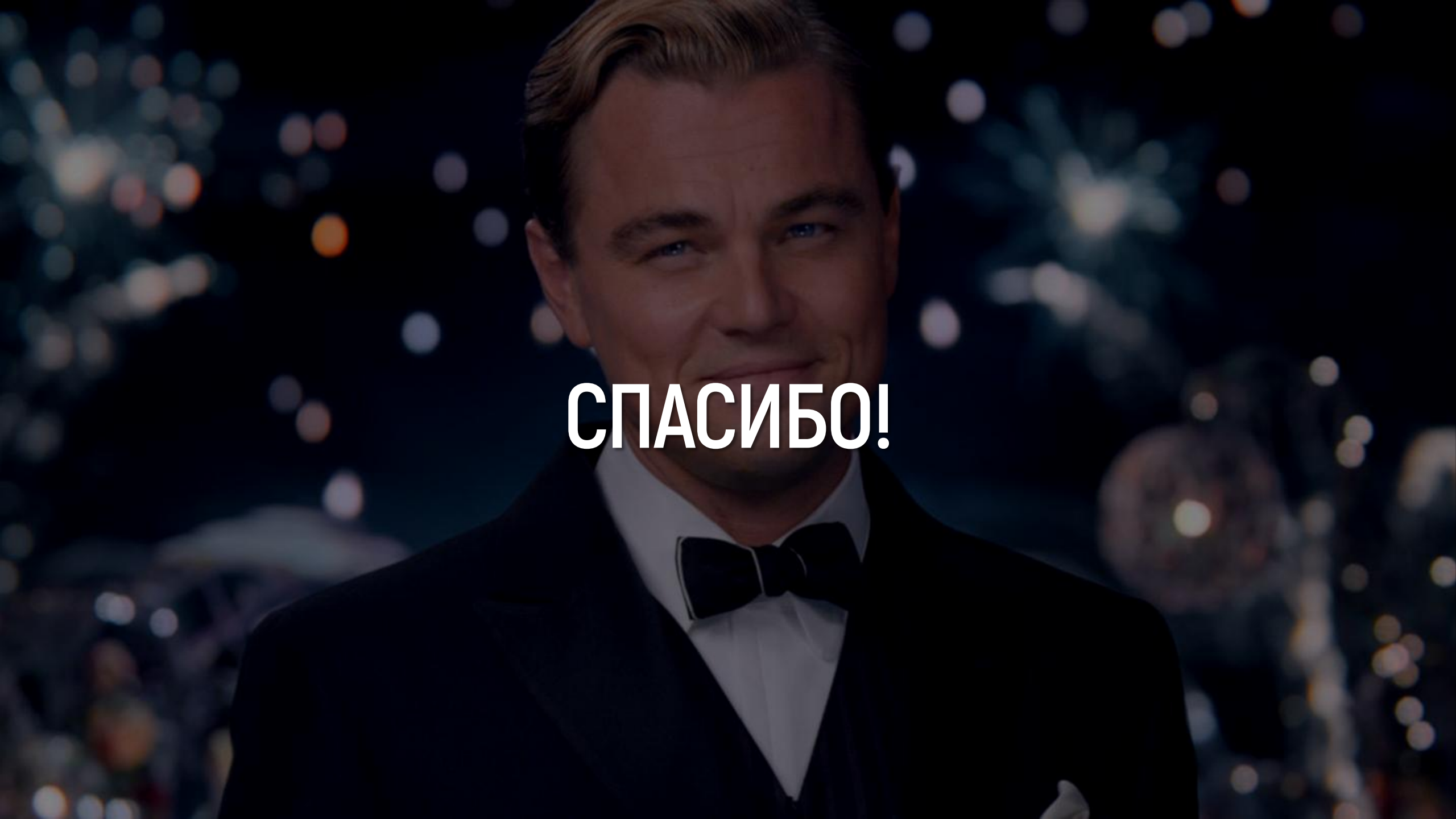
СТОЖКА, СТОЖКААА!

ВЫДЫХАЕМ

- * Постарайтесь понять идею: мы **описываем некоторый шаблон поиска** с помощью специальных языковых конструкций
- * Сегодня мы **разобрали базовые конструкции**, а на следующей лекции погрузимся уже более конкретно

ПОДВЕДЁМ ИТОГИ

- * Поиск по сложному шаблону в строке трудно описать вручную
- * Для описания некоторого языка можно использовать грамматики
- * На практике наиболее важны регулярные и контекстно-свободные грамматики
- * Они реализуются через **конечные или магазинные автоматы**
- * Регулярные выражения – это **DSL для поиска, замены и валидации строк**
- * В JS они поддерживаются через класс **RegExp** и специальный API

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!